AKADEMIA HANDLOWA NAUK STOSOWANYCH W RADOMIU

RADOM ACADEMY OF ECONOMICS

INSTRUKCJA LABORATORYJNA NUMER 1
Przedmiot: Programowanie Niskopoziomowe
Cel praktyczny zajęć: Sprawdzenie jak konstrukcje wysokiego poziomu zapisane w C# przekładają się na operacje niskopoziomowe — rejestry, flagi, skoki, adresację i kod assemblera wygenerowany przez JIT.

1. Cele laboratorium

- rozróżnianie języka maszynowego, assemblera, IL/CIL i kodu wysokiego poziomu;
- analiza podstawowych instrukcji procesora: operacje arytmetyczne, porównania, skoki, operacje bitowe;
- rozumienie roli rejestrów, flag i sposobów adresowania;
- ocena wpływu konstrukcji programu na wynikowy kod assemblera;
- weryfikacja prostych „optymalizacji” przez obserwację rzeczywistego kodu wygenerowanego z C#.

2. Wprowadzenie

Assembler to symboliczny zapis instrukcji procesora. Kod maszynowy jest wykonywany bezpośrednio przez CPU, a assembler jest jego czytelną dla człowieka reprezentacją.

W środowisku .NET kod C# nie przechodzi od razu do assemblera. Najpierw kompilator generuje IL/CIL, a następnie podczas uruchomienia JIT tłumaczy metody na kod natywny konkretnego procesora.

W tym laboratorium będziemy obserwować przede wszystkim efekt końcowy: assembler wygenerowany z kodu C# przez JIT w trakcie debugowania.

3. Schemat wykonania kodu

Kod źródłowy	Kompilacja	Kod pośredni	Uruchomienie
C#	kompilator języka	IL / CIL	JIT → kod assemblera / natywny

4. Środowisko i przygotowanie stanowiska

1. Uruchom Visual Studio i utwórz projekt typu Console App w języku C#.
2. Włącz konfigurację Debug.
3. W razie potrzeby aktywuj w ustawieniach debugera opcję Enable address-level debugging, aby korzystać z okna Disassembly.
4. Przygotuj się do pracy z breakpointami oraz przechodzenia programu krokowo klawiszami F10 i F11.

5. Odczyt assemblera wygenerowanego z kodu C#

Przepisz poniższy kod i ustaw breakpoint na linii z Console.WriteLine(result).

```
1   using System;
2
3   class Program
4   {
5       static int AbsDiff(int a, int b)
6       {
7           int diff = a - b;
8           if (diff < 0)
9               return -diff;
10          return diff;
11      }
12
13      static void Main()
14      {
15          int result = AbsDiff(3, 10);
16          Console.WriteLine(result); // breakpoint
17      }
18  }
19
```

Wykonaj następujące czynności:

5. uruchom program klawiszem F5;
6. po zatrzymaniu na breakpointcie otwórz Debug > Windows > Disassembly;
7. przechodź krokowo po instrukcjach i zanotuj, gdzie widać odejmowanie, porównanie z zerem i skok warunkowy;
8. zapisz, które fragmenty assemblera odpowiadają instrukcji if oraz zwracaniu wartości.

Pytania kontrolne
Które instrukcje odpowiadają za odejmowanie?
W jaki sposób realizowane jest if (diff < 0)?
Czy widać skok warunkowy?
Czy kod wygląda tak samo w Debug i Release?

6. Ćwiczenie 1 — Flagi i skoki warunkowe

Cel: zrozumieć, jak porównania wpływają na flagi procesora i sterowanie przebiegiem programu.

```
static int Max2(int a, int b)
{
   if (a > b)
      return a;
   return b;
}
```

Polecenia:

9. Uruchom program i otwórz Disassembly.
10. Znajdź instrukcję porównania argumentów oraz skok warunkowy.
11. Ustal, w którym miejscu wybierana jest zwracana wartość.
12. Zanotuj, dlaczego instrukcja if nie istnieje jako pojedynczy opcode CPU.

Wyzwanie: Zmodyfikuj funkcję tak, aby zwracała 1, 0 lub -1 zależnie od relacji między a i b, a następnie porównaj wygenerowany assembler z wersją podstawową.

7. Ćwiczenie 2 — Pętla, licznik i adresacja danych

Cel: zobaczyć, jak pętle i odczyt z tablicy przekładają się na operacje na rejestrach oraz pamięci.

```
static int SumArray(int[] arr)
{
   int sum = 0;
   for (int i = 0; i < arr.Length; i++)
      sum += arr[i];
   return sum;
}
```

Polecenia:

13. Uruchom funkcję dla krótkiej tablicy, np. {1, 2, 3, 4, 5}.
14. W assemblerze wskaż inicjalizację licznika, warunek zakończenia pętli i inkrementację.
15. Znajdź odczyt arr[i] i opisz, jak realizowany jest dostęp do danych.
16. Zanotuj, czy odczyt elementu wymaga obliczenia adresu i jaką rolę pełni indeks.

Wyzwanie: Napisz wersję z foreach i porównaj, czy generowany kod jest podobny oraz która wersja jest łatwiejsza do analizy.

8. Ćwiczenie 3 — Operacje bitowe i myślenie niskopoziomowe

Cel: powiązać proste operacje logiczne z naturalnym sposobem działania CPU.

```
static bool IsEven(int x)
{
    return (x & 1) == 0;
}
```

Polecenia:

17. Uruchom program i obejrzyj assembler.
18. Wskaż miejsce maskowania bitowego i porównania z zerem.
19. Wyjaśnij, dlaczego operacje bitowe są naturalne z punktu widzenia procesora.
20. Zapisz, w jaki sposób zwracana jest wartość logiczna.

Wyzwanie: Porównaj tę wersję z implementacją x % 2 == 0 i oceń, czy kod assemblera jest prostszy.

9. Ćwiczenie 4 — Minioptymalizacja i weryfikacja opcode

Cel: sprawdzić, czy intuicyjna „optymalizacja" rzeczywiście zmienia kod wynikowy.

```
static int MulBy8_A(int x) => x * 8;
static int MulBy8_B(int x) => x << 3;
```

Polecenia:

21. Podejrzyj assembler obu funkcji.
22. Porównaj, czy JIT generuje taki sam lub podobny kod.
23. Oceń, czy zapis bitowy daje realną przewagę.
24. Zanotuj wniosek: czy każdą optymalizację należy potwierdzać obserwacją lub pomiarem.

Wyzwanie: Przygotuj własny przykład „optymalizacji na skróty", a następnie sprawdź, czy końcowy kod rzeczywiście się zmienia.

10. Ćwiczenie 5 — Rejestry, flagi i przepływ sterowania

Cel: nauczyć się rekonstruować logikę programu na podstawie instrukcji niskopoziomowych.

```
static int ClampToByte(int x)
{
   if (x < 0) return 0;
   if (x > 255) return 255;
   return x;
}
```

Polecenia:

25. Przewiduj logiczny przebieg wykonania jeszcze przed otwarciem Disassembly.
26. W assemblerze wskaż oba porównania oraz miejsca zwracania wartości 0, 255 i x.
27. Narysuj prosty graf przepływu sterowania dla tej funkcji.
28. Wyjaśnij, jaką rolę pełnią flagi po każdym porównaniu.

Wyzwanie: Zmień funkcję tak, aby pracowała na typie uint i porównaj strukturę wygenerowanego kodu.

11. Zadanie kreatywne

Przeanalizuj działanie poniższej funkcji bez opisu jej przeznaczenia. Uruchom program, przejdź do widoku Disassembly w Visual Studio i na podstawie kodu assemblera oraz przepływu sterowania ustal, co robi funkcja zapisz wnioski.

```
1   static int Mystery(int[] arr)
2   {
3       int count = 0;
4
5       for (int i = 0; i < arr.Length; i++)
6       {
7           if ((arr[i] & 1) == 1)
8               count++;
9       }
10
11      return count;
12  }
```

12. Sprawozdanie

- krótki opis celu ćwiczenia;
- screen lub opis z okna Disassembly dla co najmniej dwóch funkcji;
- wskazanie instrukcji porównania, skoku, operacji na rejestrach i odczytu danych z pamięci;
- odpowiedź na pytanie: czy kod „bardziej niskopoziomowy” zapisany w C# zawsze daje lepszy assembler?;
- wnioski końcowe.